



A Machine Learning Perspective on Asset Allocation

Submitted by:
Tugce Numanoglu

This project is submitted in partial fulfillment of the requirements for the Master of
Science degree in Data Science (Non-Thesis)

Graduate School of Sciences and Engineering
Koc University

Supervised by:
Prof. Fikri Karaesmen

January 15, 2025

Acknowledgements

I would like to thank my project supervisor Prof. Fikri Karaesmen for guiding me through this project. I also want to express gratitude to my fiance Yagiz Onen supporting me at every stage and motivating me in stressful times.

Abstract

Asset allocation is an attractive problem in finance. We examine this problem from machine learning perspective. We want to see if we can improve the portfolio performance with the help of machine learning methods. We have focused on Dow-30 index US companies. Our attempt to predict monthly asset returns with machine learning resulted poorly. However, we have successfully implemented a portfolio estimation model and compared its performance w.r.t. two benchmarks and the market index. We achieve a Sharpe Ratio of 0.83 on out-of-sample data outperforming the market index and the benchmarks.

Contents

1	Introduction	4
2	Experimental Design	5
2.1	Dataset	5
2.2	Sample Splitting and Tuning via Validation	5
2.3	Mean-Variance Optimization	6
3	Methodology	7
3.1	Method-1: Plug-in Portfolio	7
3.1.1	Return Prediction	7
	i. Simple Linear Regression	7
	ii. Penalized Linear Regression	7
	iii. Gradient Boosted Regression Trees (GBRT)	8
	iv. Neural Networks	8
3.1.2	Portfolio Optimization	8
3.2	Method-2: Portfolio Estimation	9
3.2.1	Target: Optimal Weight Calculation	9
3.2.2	Weight Prediction	9
3.3	Implementation Details	9
4	Results	10
4.1	Method-1: Return Prediction	10
4.2	Portfolio Performance	10
5	Conclusion	13
	Appendices	15

1 Introduction

Asset allocation is a critical component of investment strategy in financial markets. Traditionally, it has relied on the mean-variance optimization approach outlined in Modern Portfolio Theory (MPT). However, modern machine learning algorithms have emerged as valuable tools for predicting future risks and returns based on historical data. Their capacity to process large volumes of information and identify complex patterns presents promising solutions to the limitations of traditional methods.

This report examines the application of machine learning techniques in two approaches that both utilize Modern Portfolio Theory (MPT): Method 1: "Plug-in" Portfolio and Method 2: Portfolio Estimation. We will explore various machine learning methods, including simple linear regression, gradient-boosted regression trees, and neural networks. Their performance will be evaluated over an out-of-sample period and compared to two baseline methods: the Equal Weighted Portfolio and the MVO Portfolio (max Sharpe portfolio). The objective is to provide a comprehensive assessment of how machine learning can enhance asset allocation and ultimately improve investment outcomes.

This project is heavily influenced by [Kelly und Xiu, 2023], especially sections 3-Return Prediction and 5-Optimal Portfolios in their survey. They surveyed the nascent literature on machine learning in the study of financial markets. They highlighted the best examples of what this line of research has to offer and recommended promising directions for future research.

[Gu u. a., 2020] performed a comparative analysis of machine learning methods for the canonical problem of empirical asset pricing: measuring asset risk premiums. They identified the best-performing methods (trees and neural networks). In [Gu u. a., 2020] study, trees and neural networks generate monthly out-ofsample R^2 's between 0.52% to 0.70% per month. We are inspired by their work, specifically in the return prediction section.

[Kan und Zhou, 2007] analytically derive the expected loss function associated with using sample means and the covariance matrix of returns to estimate the optimal portfolio. Their analytical results show that the standard plug-in approach that replaces the population parameters by their sample estimates can lead to very poor out-of-sample performance. They further show that with parameter uncertainty, holding the sample tangency portfolio and the riskless asset is never optimal. Their finding on the poor out-of-sample performance of "Plug-In" Portfolios aligns with the results of this project. The plug-in portfolio performed poorly in our study too.

While the finance profession has a tendency to focus on relatively low-frequency investment strategies, the computer science literature has largely focused on higher-frequency data since complex models need large amounts of data. [Hambly u. a., 2023] is one of the examples that aim to review the recent developments and use of RL approaches in finance. Although this project also aimed to asses reinforcement learning applications in finance, this wasn't possible in the project's timeline.

2 Experimental Design

It is important to explain the experimental design before going into model details. We have approached this problem from two perspectives. In the first approach, we separated the problem into two sections: return prediction, and portfolio optimization. We predicted the returns with 4 machine-learning models and selected the best one. Then, we solve the portfolio optimization problem with mean-variance optimization using the predicted returns. In the second method, we approached it as one problem, portfolio estimation. We defined an optimal portfolio and predicted it with neural networks. In both methods, the portfolios consist of weights. The weight of each individual assets is non-negative, and portfolio weights sum up to one. We compared the performance of the portfolios under the weak economic assumptions (the absence of arbitrage, frictionless trading) on out-of-sample data.

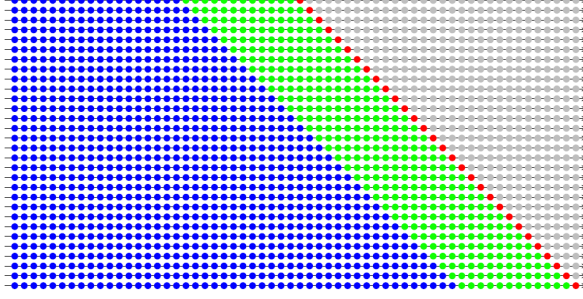
2.1 Dataset

We have obtained monthly stock prices ('Open', 'Close', 'High', 'Low', 'Volume') from Yahoo!Finance for all the firms in Dow 30 which is the second oldest stock market index in the U.S.. The data starts from January 2010 and ends in December 2024, totaling 14 years. 84 technical indicators are calculated on historical data which are pattern-based signals produced by the prices and volume. Technical analysts use indicators to predict future price movements such as the Relative Strength Index (RSI), Money Flow Index (MFI), moving average convergence divergence (MACD), and Bollinger Bands. In addition, 6 macroeconomic indicators are included in the dataset which are GDP, Unemployment Rate, Consumer Price Index, Federal Funds Rate, Interest Rate, and Federal Debt. A total of 5370 samples are present in the dataset, with 95 features. Economic indicators are updated quarterly. Linear interpolation is applied on the month level to handle the missing monthly values.

2.2 Sample Splitting and Tuning via Validation

We have implemented a time-ordered recursive design to use up-to-date data in the training for each out-of-sample data (test dataset). Figure 3, which is taken from [Kelly und Xiu, 2023], illustrates the design with the expanding training window. In Figure 3, blue dots represent training samples, green dots represent validation samples, and red dots represent test samples. Each row represents a step in the recursive design. In our case, it is a rolling window instead of an expanding window. That means training size is fixed for 7 years and validation size is fixed for 4 years. For each test sample, a new model is trained. Best hyper-parameters are chosen based on the validation score of each step. We have applied a random search on the given set of hyper-parameters. We had to keep the search shallow due to time constraints, and processing power limitations. The test set is only used for measuring out-of-sample performance.

Figure 1: Illustration of Recursive Time-ordered Cross-validation



2.3 Mean-Variance Optimization

Mean-variance optimization is based on [Markowitz, 1952]. The key insight is that by combining assets with different expected returns and volatilities, one can decide on a mathematically optimal allocation. [Martin, 2021] explains it as:

"If w is the weight vector of stocks with expected returns μ , then the portfolio return is equal to each stock's weight multiplied by its return, i.e $w^T \mu$. The portfolio risk in terms of the covariance matrix Σ is given by $w^T \Sigma w$. Portfolio optimization can then be regarded as a convex optimization problem, and a solution can be found using quadratic programming. If we denote the target return as μ^* , the precise statement of the long-only portfolio optimization problem is as follows.

$$\begin{aligned} \underset{w}{\text{minimize}} \quad & w^T \Sigma w \\ \text{subject to} \quad & w^T \mu \geq \mu^*, \\ & w^T \mathbf{1} = 1, \\ & w_i \geq 0 \end{aligned} \tag{1}$$

If we vary the target return, we will get a different set of weights (i.e a different portfolio) – the set of all these optimal portfolios is referred to as the efficient frontier."

We find the portfolio that maximizes the Sharpe ratio with the efficient frontier. The Sharpe ratio is the portfolio's return in excess of the risk-free rate, per unit risk (volatility).

$$SR = \frac{R_P - R_f}{\sigma} \tag{2}$$

The mean-variance optimization requires expected return and a risk model, which is a way of quantifying asset risk. The most commonly used risk model is the covariance matrix of assets, which describes asset volatilities and their co-dependence. We have used the covariance of assets on historical data as a risk model. For the expected return, the standard way is the mean historical return. This is defined as the Mean-Var-Portfolio in this project, and it is one of the baseline portfolios alongside the Uniform Portfolio. The baseline will be explained in detail in Section 4.

3 Methodology

3.1 Method-1: Plug-in Portfolio

Plug-in portfolio has two steps: return prediction, and portfolio optimization. We use the return predictions as expected annual returns for the portfolio optimization problem. For the return prediction, the target is defined as a monthly return which is calculated on 'Close'. Returns are defined as:

$$R_{t+1}^{(i)} = \frac{P_{t+1}^{(i)} - P_t^{(i)}}{P_t^{(i)}} \quad (3)$$

where $i = 1, \dots, N$ assets, $t = 0, \dots, T$ time points and $P_t^{(i)}$ is price of asset i at time t . For this problem, $N = 30$ since we have 30 companies in the dataset and each t corresponds to a month.

3.1.1 Return Prediction

We predicted the monthly returns with the 4 models: Linear Regression, Penalized Linear Regression, Gradient-Boosted Regression Trees, and Neural Networks. The best model is selected for the next step.

i. Simple Linear Regression

The first model is a simple linear predictive regression model. It is chosen as a reference for more complex models. Target is the return of a stock i at a time t : $\hat{y}_{t+1}^{(i)} = R_{t+1}^{(i)}$. Linear regression coefficients are defined as the vector $w_t = (w_t^{(1)}, w_t^{(2)}, \dots, w_t^{(p)})$ with p as the feature size 95, and w_t^0 as intercept (if it is selected in the hyper-parameter search). With X_t as the feature set, the linear regression formula is:

$$\hat{y}_{t+1} = w_t^0 + w_t X_t + \epsilon_t \quad (4)$$

The model minimizes Ordinary least squares (OLS): the residual sum of squares between target and the predictions, as follows:

$$\underset{w}{\text{minimize}} \quad \|\mathbf{X}^{(i)} \mathbf{w} - \mathbf{y}\|_2^2 \quad (5)$$

ii. Penalized Linear Regression

We apply a penalization term to the linear regression model. Penalized linear Regression solves the problem of minimizing Linear least Squares with L2 regularization. The closed-form solution of the model is as follows:

$$\underset{w}{\text{minimize}} \quad \|\mathbf{X}^{(i)} \mathbf{w} - \mathbf{y}\|_2^2 + \alpha \|\mathbf{w}\|_2^2 \quad (6)$$

$\alpha \geq 0$ controls the amount of shrinkage that helps prevent coefficients from becoming unduly large in magnitude. We optimized the tuning parameter α on the validation set within the given set (0.5, 3, 4), and mostly 4 is selected.

iii. Gradient Boosted Regression Trees (GBRT)

Non-linearity is introduced by gradient-boosted regression Trees. In GBRT, we combine many weak learners to come up with one strong learner. The weak learners are the individual decision trees, they are built sequentially. Each new tree is trained to correct the errors made by the previous trees. The hyper-parameters are n-estimators (the number of trees used in the model), and max-depth (the maximum depth of the individual regression estimators). Unfortunately, GBRT is too slow, and we didn't have time to do hyper-parameter search thoroughly. The model minimizes the same loss function, residual sum of squares (RSS), with gradient descent. The RSS is as follows:

$$\text{RSS} = \sum_{i=1}^n (y_i - \hat{y}_i)^2 \quad (7)$$

iv. Neural Networks

For the last model, we have chosen a more advanced nonlinear method, the artificial neural network. The model is a traditional "feed-forward" network. The loss function is defined as mean squared error, and the optimizer is selected as Adam Optimizer, which is one of the stochastic gradient descent methods. $Q(w)$ is the loss function :

$$w := w - \alpha \nabla Q(w) \quad (8)$$

The model has 2 fully connected hidden layers with 32, and 16 neurons. They have the nonlinear activation function as the rectified linear unit (ReLU):

$$\text{ReLU}(x) = \max(0, x) \quad (9)$$

We have the liner activation function at the output layer, as:

$$\text{Linear}(x) = x \quad (10)$$

We have applied a couple of hyper-parameters to prevent over-fitting: L2 regularization that penalizes large weight parameters, learning rate shrinkage that shrinks the learning rate toward zero as the gradient approaches zero (to prevent the model from fitting the noise), early stopping that stops the training when the validation error starts to increase, and batch normalization that normalize of the layers' inputs by re-centering and re-scaling for more stable training through.

3.1.2 Portfolio Optimization

In this step, we perform portfolio optimization with mean-variance optimization(MVO). The portfolio weights $w_t = (w_t^{(1)}, w_t^{(2)}, \dots, w_t^{(30)})$ is calculated via MVO at each month. The optimizer needs 2 input data: we use the previous step's result, expected returns the annual expected returns (multiplied by 12), and the covariance of the assets returns on the last 12 months as risk model.

3.2 Method-2: Portfolio Estimation

In this section, we train a neural network model to predict the optimal portfolio weights. The optimal portfolio is calculated via MVO. Then, NN models are trained with the optimal weights as targets.

3.2.1 Target: Optimal Weight Calculation

The portfolio weight vector is defined as $w_t = (w_t^{(1)}, w_t^{(2)}, \dots, w_t^{(30)})$ same as Method-1 portfolio weights. We take the next month's return as the annual expected return (multiplied by 12), and the covariance of the assets returns on the last 12 months as a risk model like the Method-1. We have proved the performance of the optimal portfolio on out-of-sample data, which has a Sharpe ratio of 5.71 (above 3 considered excellent) and an annual return of 213%.

3.2.2 Weight Prediction

In this problem, the loss function is defined as mean squared error, and the optimizer is selected as Adam Optimizer. The model starts with 1-dimensional convolution (the filter size 32, and the kernel size is 3). It is followed by a max-pooling layer and a flatten layer. The remaining part of the model same as the Method-1 neural network. The only difference is the output layer. This time the output layer has 30 neurons and a Softmax function as the activation function to have non-negative weights which sum up to 1. The Softmax function is:

$$\text{softmax}(x_i) = \frac{e^{x_i}}{\sum_{j=1}^n e^{x_j}} \quad (11)$$

We have applied the same hyper-parameters as the Method-1 neural network to overcome over-fitting: L2 regularization, learning rate shrinkage, early stopping, and batch normalization. In addition to these L1 regularization is applied too, which penalizes the high weights like L2 regularization. L1 regularization has the advantage of making the models robust to outliers.

3.3 Implementation Details

This project is implemented with Python since it is widely used in the machine learning area. "yfinance" to download market data from Yahoo!Finance's API. Economic data is fetched from FRED (Federal Reserve Economic Data) with "pandas-datareader" which provides API access to many data sources. Technical indicators are calculated with "stockstats" which provides stock statistics/indicators support. "PyPortfolioOpt" is used for mean-variance optimization (MVO). "Scikit-learn" [Pedregosa et al., 2011] library is used for machine learning models, standard normalization, and model performance metrics. "tensorflow" is used for neural network models. Financial metrics are calculated by "empyrical" library. "keras-tuner" is used for hyper-parameter search of the NN models.

4 Results

We will present the results under 2 sections: Method-1 Return Prediction Performance, and Portfolio Performance, which includes Method-1 portfolio, Method-2 portfolio, and two baselines.

4.1 Method-1: Return Prediction

Return prediction is the subproblem of the Plug-in Portfolio. We want to demonstrate the return prediction results separately since model selection is performed at this stage. We can see that the best performed model is the neural network model from Table 1. Unfortunately, the out-of-sample R^2 score of the return predictions is still negative. We can note here that the training R^2 score of the NN model was 4% so the model was learning the training set. In [Gu et al., 2020], they achieved R^2 score of the out-of-sample ranging from 0.52% to 0.70% with neural networks and tree models. This also proves that the monthly return prediction problem has low R^2 scores in the literature.

Table 1: Monthly out-of-sample Asset Return Prediction Performance

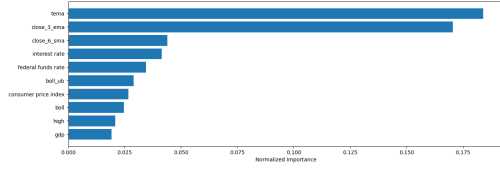
Model	RMSE	Percentage R^2
Linear Regression	0.096	-38
Penalized Linear Regression	0.092	-25
GBRT	0.09	-20
Neural Network	0.085	-7

We also check the feature importance of the return predictors with Shapley values. They are calculated using a game-theoretic approach to determine the contribution of each feature to the prediction of a machine learning model. Figure 2 shows normalized values for interpretability. We can see that economic indicators are highly important features. Interest Rate is at the top for the NN Model and GBRT Model. Unemployment Rate is also in common for both models. The dominant technical indicators common in NN and GBRT models are ADX (Average Directional Movement Index), Close price SMA (Simple Moving Average), and High price also High Delta (change w.r.t. previous period). The linear and the penalized linear models have the Consumer Price Index as the common indicator, and the linear model has TEMA (Another Triple Exponential Moving Average) as the most important feature.

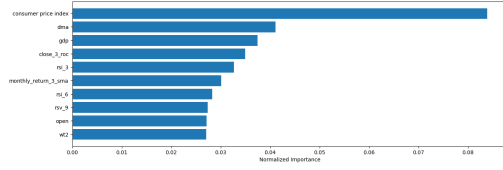
4.2 Portfolio Performance

We have defined 2 baseline portfolios. The first baseline is the Uniform Portfolio (it has uniform weights), and the second portfolio is the Mean-Var Portfolio whose annual return is the historical mean of 12 months (multiplied by 12), and variance is also calculated in the same way as Method-1 and Method-2 (on past 12 months). We also have the DOW-30 portfolio itself which represents the market. Except for the Dow-30 portfolio (it is managed by S&P Dow Jones Indices LLC), the weights of our portfolios are updated monthly. We assumed no transaction costs were involved except DOW-30. We assumed to have 1000000\$ capital at the beginning, and we calculated the monthly capital with changing weights.

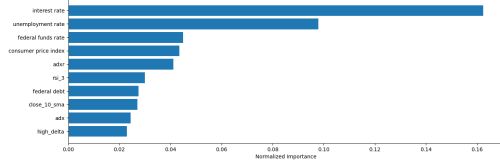
Figure 2: Return Prediction Models' Feature Importance



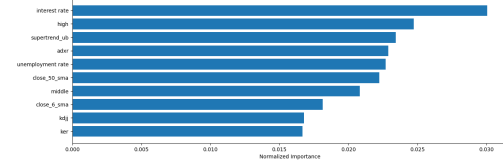
(a) Linear Model



(b) Penalized Linear Model



(c) GBRT Model



(d) Neural Network Model

The top trio: the Uniform, Mean-Var, and Method-2 portfolios achieved around 0.4 cumulative return. In the first period of 2024, Method-2 outperformed the other portfolios significantly, but at other times, it performed similarly in the top trio. Even if the Uniform Portfolio is a simple approach it is still in the top 3 best portfolios in the backtest. However, it has never outperformed the Method-2 portfolio in time. The Mean-Var Portfolio outperformed the all in the beginning period of 2022 and underperformed in the top trio at the beginning of 2024. At other times, It has performed similarly in the top trio. Unfortunately, the Method-1 portfolio has zero cumulative return, so this approach was unsuccessful since the aim was to get positive returns. It is also important to note that the market index, Dow-30 is outperformed by the top trio.

Figure 3: Backtest Cumulative Return based on the Data from 2022-01-01 to 2024-10-01

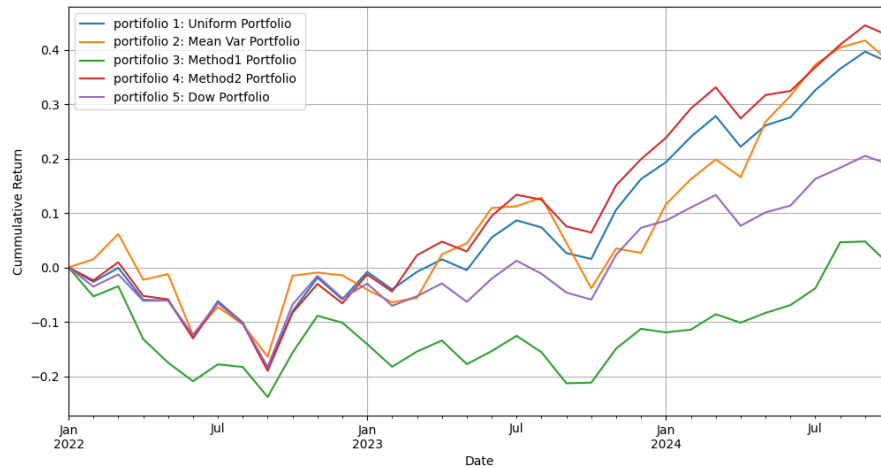


Table 2 shows the performance metrics of all the portfolios. We can see that the Method-2 portfolio outperforms others on annual return and cumulative return. The majority of the portfolios have 0.17 annual volatility except the mean-variance portfolio which means that it is more risky than others. The Sharpe ratio and the Calmar Ratio are both risk-adjusted performance metrics. The Calmar Ratio shows the annual return over the maximum drawdown (The Sharpe ratio is explained in the MVO section). Method 2 outperforms others on both of these metrics. Maximum drawdown is the largest percentage decline from the investment's peak value to its subsequent lowest point. The Dow portfolio has the lowest maximum dropdown. The Sortino Ratio is a variation of the Sharpe Ratio that focuses on downside risk, providing a measure of risk-adjusted return that only considers harmful volatility. Method-2 has the highest Sortino Ratio. The Tail Ratio is a risk metric used to evaluate the likelihood of extreme returns in an investment portfolio. Dow has the highest tail ratio. Method-1 with a value (1.23) more than 1 indicates that the portfolio has a higher average return in the right tail (positive returns), but this is far from what happened in the backtest period. Alpha measures the performance of an investment relative to a benchmark index, which is the Dow. Positive Alpha indicates the investment has outperformed the market. Except for Method-1 all of the portfolios outperformed the market. Method-2, and the Mean-Var have the highest Alpha values. Beta measures the volatility relative to the overall market. The Uniform portfolio and Method-2 with a value (1.02) higher than 1 indicates that they are more volatile than the market. On the other hand, the other portfolios are less volatile than the Dow.

Table 2: Monthly Portfolio Performances

Portfolio	Dow	Uniform	Mean Var	Method1	Method2
Annual Return	0.06	0.12	0.12	0.0	0.13
Cumulative Returns	0.19	0.38	0.38	0.0	0.42
Annual Volatility	0.17	0.17	0.2	0.17	0.17
Sharpe Ratio	0.46	0.76	0.67	0.09	0.83
Calmar Ratio	0.35	0.64	0.56	0.01	0.67
Stability	0.72	0.83	0.69	0.25	0.86
Max Drawdown	-0.18	-0.19	-0.21	-0.24	-0.2
Sortino Ratio	0.77	1.3	1.14	0.14	1.43
Tail Ratio	1.33	1.27	1.09	1.23	1.16
Alpha	0.0	0.05	0.07	-0.05	0.07
Beta	1.0	1.02	0.96	0.86	1.02

5 Conclusion

We have performed a comparative analysis of two machine learning approaches to asset allocation. We proved that machine learning models can improve asset allocation performance. It is important to note that we assumed frictionless trading (no transaction costs) for simplicity which means that our best portfolio might not outperform the market in real life. If there are obvious patterns known by the market, then the exploitation of the pattern would increase asset prices, and the pattern would be no longer useful. So, recognizable patterns are useful as long as they are not available on the market.

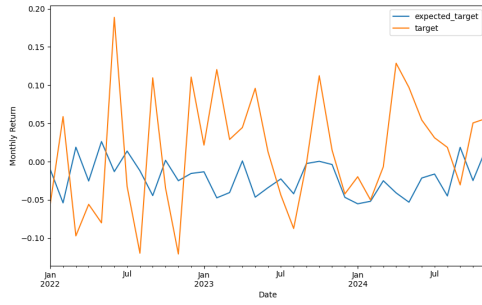
Neural networks are the best-performed models in the return prediction since they can catch non-linear interactions. However, the results weren't successful since monthly asset prices have a low signal-to-noise ratio, and this prevents the model's learning ability. We tried to avoid it with over-fit preventative methods like regularizes, early stops, etc. We see that Method-1, Plug-In portfolio performed poorly, which is worse than the benchmarks. This is expected because of the negative R^2 score (-7%) of the out-of-sample return prediction. Method-2, portfolio estimation successfully outperformed the benchmarks with the Sharpe value of 0.83. Our results are promising in investigating the performance of more complex models such as LSTM or Reinforcement Learning. In the future, we would like to take into account the transaction costs, and we would like to compare our findings with LSTM and RL models.

References

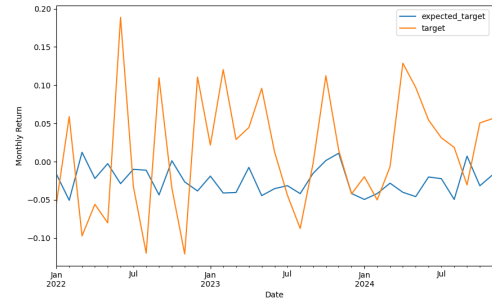
- [Gu u. a. 2020] GU, Shihao ; KELLY, Bryan ; XIU, Dacheng: Empirical Asset Pricing via Machine Learning. In: *The Review of Financial Studies* 33 (2020), 02, Nr. 5, S. 2223–2273. – URL <https://doi.org/10.1093/rfs/hhaa009>. – ISSN 0893-9454
- [Hambly u. a. 2023] HAMBLY, Ben ; XU, Renyuan ; YANG, Huining: Recent advances in reinforcement learning in finance. In: *Mathematical Finance* 33 (2023), Nr. 3, S. 437–503
- [Kan und Zhou 2007] KAN, Raymond ; ZHOU, Guofu: Optimal Portfolio Choice with Parameter Uncertainty. In: *Journal of Financial and Quantitative Analysis* 42 (2007), Nr. 3, S. 621–656
- [Kelly und Xiu 2023] KELLY, Bryan ; XIU, Dacheng: Financial Machine Learning. In: *Foundations and Trends® in Finance* 13 (2023), Nr. 3-4, S. 205–363. – URL <http://dx.doi.org/10.1561/05000000064>. – ISSN 1567-2395
- [Markowitz 1952] MARKOWITZ, Harry: PORTFOLIO SELECTION. In: *The Journal of Finance* 7 (1952), Nr. 1, S. 77–91. – URL <https://onlinelibrary.wiley.com/doi/abs/10.1111/j.1540-6261.1952.tb01525.x>
- [Martin 2021] MARTIN, Robert A.: PyPortfolioOpt: portfolio optimization in Python. In: *Journal of Open Source Software* 6 (2021), Nr. 61, S. 3066. – URL <https://doi.org/10.21105/joss.03066>
- [Pedregosa u. a. 2011] PEDREGOSA, F. ; VAROQUAUX, G. ; GRAMFORT, A. ; MICHEL, V. ; THIRION, B. ; GRISEL, O. ; BLONDEL, M. ; PRETTENHOFER, P. ; WEISS, R. ; DUBOURG, V. ; VANDERPLAS, J. ; PASSOS, A. ; COURNAPEAU, D. ; BRUCHER, M. ; PERROT, M. ; DUCHESNAY, E.: Scikit-learn: Machine Learning in Python. In: *Journal of Machine Learning Research* 12 (2011), S. 2825–2830

Appendices

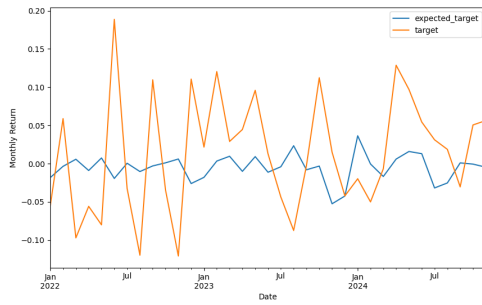
Figure 4: Method-1:Monthly Return Predictions of the Models for the asset APPLE



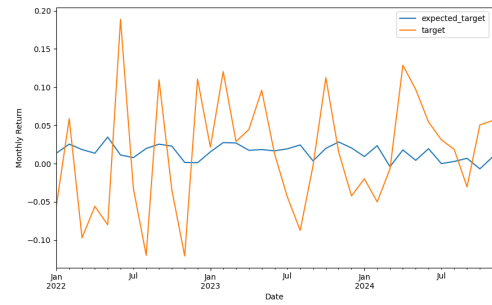
(a) Linear Model



(b) Penalized Linear Model



(c) GBRT Model



(d) Neural Network Model